

## 08. profile YAML 読込と検証

solar\_ws0129-main

2026-07-29

### Contents

|                                           |          |
|-------------------------------------------|----------|
| <b>1 対象</b>                               | <b>3</b> |
| <b>2 このファイルを読む理由</b>                      | <b>3</b> |
| <b>3 呼び出し関係</b>                           | <b>3</b> |
| 3.1 どこから呼ばれるか                             | 3        |
| 3.2 次に読むと理解が進むファイル                        | 3        |
| 3.3 同一リポジトリ内の主要依存                         | 3        |
| <b>4 ソース冒頭の把握</b>                         | <b>3</b> |
| 4.1 代表コード断片                               | 4        |
| <b>5 主要構造の読み取り</b>                        | <b>4</b> |
| 5.1 主要クラス・関数                              | 4        |
| 5.2 ファイルを上から読んだときの定義順                     | 4        |
| 5.3 import 群の詳細                           | 5        |
| <b>6 このファイルを読む前に必要な基礎知識</b>               | <b>5</b> |
| 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する            | 5        |
| 6.2 名前、代入、型、参照                            | 6        |
| 6.3 関数、引数、戻り値、スコープ                        | 6        |
| 6.4 module、package、import、console_scripts | 6        |
| 6.5 例外、try/finally、with、資源解放              | 7        |
| 6.6 制御、状態、入力、モデル予測制御 MPC                  | 7        |
| <b>7 関数・クラスを上から順に解説</b>                   | <b>7</b> |
| 7.1 L14 関数 require_csv_data_rows          | 7        |
| 7.2 L37 関数 _resolve_profile_path          | 9        |
| 7.3 L51 関数 resolve_relative_path          | 10       |
| 7.4 L68 関数 load_profile                   | 11       |
| 7.5 L77 関数 merged_dict                    | 12       |
| 7.6 L78 関数 merged_dict.merge              | 13       |
| 7.7 L93 関数 get_section                    | 14       |

|                                             |    |
|---------------------------------------------|----|
| 7.8 L108 関数 <code>get_path</code> . . . . . | 15 |
| 8 処理の流れ                                     | 16 |
| 9 このファイルの位置づけ                               | 16 |

## 1 対象

| 項目          | 内容                                                               |
|-------------|------------------------------------------------------------------|
| ファイル        | mpc_solarcar/solar_profile.py                                    |
| ソース SHA-256 | 342c5f001c7ae09a4ccfafe19e30706d124ac8b044852c365ca03ee664af00e7 |
| 種別          | Python                                                           |
| 区分          | config                                                           |
| 役割          | profile.yaml をロードし、セクション取得、相対パス解決、CSV 最低品質検査を行う。                 |
| 起動文脈        | launch、offline simulation、identification の共通設定入口。                |

## 2 このファイルを読む理由

profile.yaml をロードし、セクション取得、相対パス解決、CSV 最低品質検査を行う。

- load\_profile が YAML 全体を返す。
- get\_path が profile 基準で実ファイルパスへ変換する。
- require\_csv\_data\_rows が空テンプレートと実データを区別する。

## 3 呼び出し関係

### 3.1 どこから呼ばれるか

- live\_launch.py
- solarcar\_sim.launch.py
- solar\_state\_node.py
- 多数の scripts

### 3.2 次に読むと理解が進むファイル

- mpc\_solarcar/path\_utils.py

### 3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

## 4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

## 4.1 代表コード断片

```
ROOT = Path(__file__).resolve().parents[1]

def require_csv_data_rows(
    path: str | Path,
    *,
    label: str,
    required_columns: tuple[str, ...] = (),
) -> Path:
    resolved = Path(path).expanduser().resolve()
    if not resolved.is_file():
        raise FileNotFoundError(f"{label} CSV was not found: {resolved}")
    with resolved.open("r", encoding="utf-8-sig", newline="") as stream:
        reader = csv.DictReader(stream)
        header = tuple(reader.fieldnames or ())
        missing = [column for column in required_columns if column not in header]
        if missing:
            raise ValueError(f"{label} CSV is missing columns {missing}: {resolved}")
        if next(reader, None) is None:
            raise ValueError(
                f"{label} CSV has a header but no data rows: {resolved}. "
                "Fill the template or select an identified vehicle profile before launch."
```

## 5 主要構造の読み取り

主要関数は `require_csv_data_rows`, `resolve_relative_path`, `load_profile`, `merged_dict`, `get_section`, `get_path`。

### 5.1 主要クラス・関数

| 行   | 種別       | 名前                                 | 補足                                    |
|-----|----------|------------------------------------|---------------------------------------|
| 14  | function | <code>require_csv_data_rows</code> | args: path                            |
| 37  | function | <code>_resolve_profile_path</code> | args: path_like                       |
| 51  | function | <code>resolve_relative_path</code> | args: base_dir, path_like             |
| 68  | function | <code>load_profile</code>          | args: path_like                       |
| 77  | function | <code>merged_dict</code>           |                                       |
| 78  | function | <code>_merge</code>                | args: dst, src                        |
| 93  | function | <code>get_section</code>           | args: cfg, key, default               |
| 108 | function | <code>get_path</code>              | args: cfg, profile_path, key, default |

### 5.2 ファイルを上から読んだときの定義順

| 行  | 内容                                                                |
|----|-------------------------------------------------------------------|
| 11 | ROOT に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。 |
| 14 | 関数 <code>require_csv_data_rows</code> を定義する。                      |
| 37 | 関数 <code>_resolve_profile_path</code> を定義する。                      |
| 51 | 関数 <code>resolve_relative_path</code> を定義する。                      |
| 68 | 関数 <code>load_profile</code> を定義する。                               |
| 77 | 関数 <code>merged_dict</code> を定義する。                                |

93      関数 `get_section` を定義する。  
108     関数 `get_path` を定義する。

### 5.3 import 群の詳細

| 行 | import 文                                             | このファイルで読む理由                                                                                                                  |
|---|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>from __future__<br/>_import annotations</code> | 型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。                                                 |
| 3 | <code>import copy</code>                             | 設定辞書や <code>payload</code> を安全に複製するため。このファイル内での主な使用位置は L81, L83, L99, L102, L104, L105。                                      |
| 4 | <code>import csv</code>                              | CSV の逐次読込・逐次書込を行うため。このファイル内での主な使用位置は L24。                                                                                    |
| 5 | <code>from pathlib import Path</code>                | ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L11, L15, L19, L20, L37, L38, L42, L51, ...。                                            |
| 6 | <code>from typing import Any</code>                  | <code>typing</code> から <code>Any</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L86, L93, L94。                              |
| 8 | <code>import yaml</code>                             | <code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L71。 |

## 6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

### 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

### 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

### 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘\*args’、‘\*\*kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

### 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.4 module、package、import、console\_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console\_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

### 根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

## 6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

## 6.6 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を  $x$ 、操作入力を  $u$ 、外乱・予報を  $w$  とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’ と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から  $N$  ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

### 根拠資料

- SciPy 公式: [scipy.optimize.minimize](https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html)

## 7 関数・クラスを上から順に解説

### 7.1 L14 関数 require\_csv\_data\_rows

- 行範囲: L14–L34
- 所属: module 直下
- 定義: `require_csv_data_rows(path:str|Path,*,label:str,required_columns:tuple[str,...]=())->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: DictReader, FileNotFoundError, Path, ValueError, expanduser, is\_file, next, open, resolve, tuple
- 戻り値の要点: resolved
- 主なローカル代入名: column, header, missing, reader, resolved, stream
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError(f'{label} CSV was not found: {resolved}')`, `ValueError(f'{label} CSV has a header but no data rows: {resolved}. Fill the template or select an identified vehicle profile before launch.')`, `ValueError(f'{label} CSV is missing columns {missing}: {resolved}')`

- 制御構造の規模: 条件分岐 3、ループ 0、try 0

## この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して collection を作る。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

## 上から順の処理

1. resolved に Path(path).expanduser().resolve() の結果を代入する。
2. 条件 not resolved.is\_file() を判定し、真なら内部処理を行う。
3. FileNotFoundError(f'{label} CSV was not found: {resolved}') を送出する。
4. with 文で resolved.open('r', encoding='utf-8-sig', newline='') を管理しながら処理する。
5. reader に csv.DictReader(stream) の結果を代入する。
6. header に tuple(reader.fieldnames or ()) の結果を代入する。
7. missing に [column for column in required\_columns if column not in header] の結果を代入する。
8. 条件 missing を判定し、真なら内部処理を行う。
9. ValueError(f'{label} CSV is missing columns {missing}: {resolved}') を送出する。
10. 条件 next(reader, None) is None を判定し、真なら内部処理を行う。
11. ValueError(f'{label} CSV has a header but no data rows: {resolved}. Fill the template or select an identified vehicle profile before launch.') を送出する。
12. resolved を返す。

## 代表コード断片

```
def require_csv_data_rows(
    path: str | Path,
    *,
    label: str,
    required_columns: tuple[str, ...] = (),
) -> Path:
    resolved = Path(path).expanduser().resolve()
    if not resolved.is_file():
        raise FileNotFoundError(f"{label} CSV was not found: {resolved}")
    with resolved.open("r", encoding="utf-8-sig", newline="") as stream:
        reader = csv.DictReader(stream)
        header = tuple(reader.fieldnames or ())
        missing = [column for column in required_columns if column not in header]
        if missing:
            raise ValueError(f"{label} CSV is missing columns {missing}: {resolved}")
        if next(reader, None) is None:
            raise ValueError(
                f"{label} CSV has a header but no data rows: {resolved}. "
                "Fill the template or select an identified vehicle profile before launch."
            )
    return resolved
```

## 7.2 L37 関数 `_resolve_profile_path`

- 行範囲: L37–L48
- 所属: module 直下
- 定義: `_resolve_profile_path(path_like:str|Path)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `cwd`, `exists`, `expanduser`, `is_absolute`, `resolve`, `str`, `strip`
- 戻り値の要点: `candidates[-1] / raw.resolve() / candidate`
- 主なローカル代入名: `candidate`, `candidates`, `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. `raw` に `Path(str(path_like or "").strip()).expanduser()` の結果を代入する。
2. 条件 `raw.is_absolute()` を判定し、真なら内部処理を行う。
3. `raw.resolve()` を返す。
4. `candidates` に `[(Path.cwd() / raw).resolve(), (ROOT / raw).resolve()]` の結果を代入する。
5. `candidates` を順に走査し、各要素を `candidate` に入れて処理する。
6. 条件 `candidate.exists()` を判定し、真なら内部処理を行う。
7. `candidate` を返す。
8. `candidates[-1]` を返す。

### 代表コード断片

```
def _resolve_profile_path(path_like: str | Path) -> Path:
    raw = Path(str(path_like or "").strip()).expanduser()
    if raw.is_absolute():
        return raw.resolve()
    candidates = [
        (Path.cwd() / raw).resolve(),
        (ROOT / raw).resolve(),
    ]
    for candidate in candidates:
        if candidate.exists():
            return candidate
    return candidates[-1]
```

### 7.3 L51 関数 `resolve_relative_path`

- 行範囲: L51–L65
- 所属: module 直下
- 定義: `resolve_relative_path(base_dir:str|Path,path_like:str|Path)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `exists`, `expanduser`, `is_absolute`, `resolve`, `str`, `strip`
- 戻り値の要点: `str(candidate) / ” / str(path.resolve()) / str(candidate)`
- 主なローカル代入名: `base`, `candidate`, `path`, `raw`, `repo_candidate`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、`try` 0

#### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

#### 上から順の処理

1. `raw` に `str(path_like or ”).strip()` の結果を代入する。
2. 条件 `not raw` を判定し、真なら内部処理を行う。
3. `”` を返す。
4. `path` に `Path(raw).expanduser()` の結果を代入する。
5. 条件 `path.is_absolute()` を判定し、真なら内部処理を行う。
6. `str(path.resolve())` を返す。
7. `base` に `Path(base_dir).resolve()` の結果を代入する。
8. `candidate` に `(base / path).resolve()` の結果を代入する。
9. 条件 `candidate.exists()` を判定し、真なら内部処理を行う。
10. `str(candidate)` を返す。
11. `repo_candidate` に `(ROOT / path).resolve()` の結果を代入する。
12. 条件 `repo_candidate.exists()` を判定し、真なら内部処理を行う。
13. `str(repo_candidate)` を返す。
14. `str(candidate)` を返す。

#### 代表コード断片

```
def resolve_relative_path(base_dir: str | Path, path_like: str | Path) -> str:
    raw = str(path_like or "").strip()
    if not raw:
        return ""
    path = Path(raw).expanduser()
    if path.is_absolute():
        return str(path.resolve())
    base = Path(base_dir).resolve()
    candidate = (base / path).resolve()
    if candidate.exists():
        return str(candidate)
    repo_candidate = (ROOT / path).resolve()
    if repo_candidate.exists():
        return str(repo_candidate)
    return str(candidate)
```

## 7.4 L68 関数 load\_profile

- 行範囲: L68–L74
- 所属: module 直下
- 定義: load\_profile(path\_like:str|Path)->tuple[Path,dict]
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: ValueError, \_resolve\_profile\_path, isinstance, open, safe\_load
- 戻り値の要点: (profile\_path, cfg)
- 主なローカル代入名: cfg, f, profile\_path
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: ValueError(f'profile must be a mapping: {profile\_path}')
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

### この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- with 文は context manager に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

### 上から順の処理

1. profile\_path に \_resolve\_profile\_path(path\_like) の結果を代入する。
2. with 文で profile\_path.open('r', encoding='utf-8') を管理しながら処理する。
3. cfg に yaml.safe\_load(f) or {} の結果を代入する。
4. 条件 not isinstance(cfg, dict) を判定し、真なら内部処理を行う。
5. ValueError(f'profile must be a mapping: {profile\_path}')
6. (profile\_path, cfg) を返す。

#### 代表コード断片

```
def load_profile(path_like: str | Path) -> tuple[Path, dict]:
    profile_path = _resolve_profile_path(path_like)
    with profile_path.open("r", encoding="utf-8") as f:
        cfg = yaml.safe_load(f) or {}
    if not isinstance(cfg, dict):
        raise ValueError(f"profile must be a mapping: {profile_path}")
    return profile_path, cfg
```

## 7.5 L77 関数 merged\_dict

- 行範囲: L77-L90
- 所属: module 直下
- 定義: merged\_dict(\*payloads:dict|None)->dict
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: \_merge, deepcopy, get, isinstance, items
- 戻り値の要点: out / dst
- 主なローカル代入名: key, out, payload, value
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 2、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. 関数 \_merge を定義する。
2. out に {} を代入する。
3. payloads を順に走査し、各要素を payload に入れて処理する。
4. 条件 isinstance(payload, dict) を判定し、真なら内部処理を行う。
5. out に \_merge(out, payload) の結果を代入する。
6. out を返す。

#### 代表コード断片

```
def merged_dict(*payloads: dict | None) -> dict:
    def _merge(dst: dict, src: dict) -> dict:
        for key, value in (src or {}).items():
            if isinstance(value, dict) and isinstance(dst.get(key), dict):
                dst[key] = _merge(copy.deepcopy(dst[key]), value)
            else:
                dst[key] = copy.deepcopy(value)
    return dst
```

```

out: dict[str, Any] = {}
for payload in payloads:
    if isinstance(payload, dict):
        out = _merge(out, payload)
return out

```

## 7.6 L78 関数 merged\_dict.\_merge

- 行範囲: L78–L84
- 所属: merged\_dict
- 定義: `_merge(dst:dict,src:dict)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_merge, deepcopy, get, isinstance, items`
- 戻り値の要点: `dst`
- 主なローカル代入名: `key, value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. `(src or {}).items()` を順に走査し、各要素を `(key, value)` に入れて処理する。
2. 条件 `isinstance(value, dict) and isinstance(dst.get(key), dict)` を判定し、真なら内部処理を行う。
3. `dst[key]` に `_merge(copy.deepcopy(dst[key]), value)` の結果を代入する。
4. 上の条件が偽の場合:
5. `dst[key]` に `copy.deepcopy(value)` の結果を代入する。
6. `dst` を返す。

### 代表コード断片

```

def _merge(dst: dict, src: dict) -> dict:
    for key, value in (src or {}).items():
        if isinstance(value, dict) and isinstance(dst.get(key), dict):
            dst[key] = _merge(copy.deepcopy(dst[key]), value)
        else:
            dst[key] = copy.deepcopy(value)
    return dst

```

## 7.7 L93 関数 `get_section`

- 行範囲: L93–L105
- 所属: module 直下
- 定義: `get_section(cfg:dict,key:str,default:Any=None)->Any`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `deepcopy`, `get`, `isinstance`, `split`, `str`
- 戻り値の要点: `copy.deepcopy(current) / copy.deepcopy(default) if default is not None else {} / copy.deepcopy(default) if default is not None else {} / copy.deepcopy(default) if default is not None else {}`
- 主なローカル代入名: `current`, `part`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 1、try 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. `current` に `cfg` を代入する。
2. `str(key or "").split('.')` を順に走査し、各要素を `part` に入れて処理する。
3. 条件 `not part` を判定し、真なら内部処理を行う。
4. `Continue` 文を実行する。
5. 条件 `not isinstance(current, dict)` を判定し、真なら内部処理を行う。
6. `copy.deepcopy(default) if default is not None else {}` を返す。
7. `current` に `current.get(part)` の結果を代入する。
8. 条件 `current is None` を判定し、真なら内部処理を行う。
9. `copy.deepcopy(default) if default is not None else {}` を返す。
10. 条件 `current is None` を判定し、真なら内部処理を行う。
11. `copy.deepcopy(default) if default is not None else {}` を返す。
12. `copy.deepcopy(current)` を返す。

#### 代表コード断片

```
def get_section(cfg: dict, key: str, default: Any = None) -> Any:
    current: Any = cfg
    for part in str(key or "").split("."):
        if not part:
            continue
        if not isinstance(current, dict):
            return copy.deepcopy(default) if default is not None else {}
        current = current.get(part)
        if current is None:
            return copy.deepcopy(default) if default is not None else {}
    if current is None:
        return copy.deepcopy(default) if default is not None else {}
    return copy.deepcopy(current)
```

## 7.8 L108 関数 get\_path

- 行範囲: L108–L118
- 所属: module 直下
- 定義: `get_path(cfg:dict,profile_path:str|Path,key:str,default:str='')->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `get`, `get_section`, `isinstance`, `resolve`, `resolve_relative_path`, `str`, `strip`
- 戻り値の要点: `resolve_relative_path(profile_dir, raw)`
- 主なローカル代入名: `paths_cfg`, `profile_dir`, `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. `profile_dir` に `Path(profile_path).resolve().parent` の結果を代入する。
2. `raw` に” の結果を代入する。
3. `paths_cfg` に `cfg.get('paths', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
4. 条件 `isinstance(paths_cfg, dict)` を判定し、真なら内部処理を行う。
5. `raw` に `str(paths_cfg.get(key, "") or "").strip()` の結果を代入する。
6. 条件 `not raw` を判定し、真なら内部処理を行う。
7. `raw` に `str(get_section(cfg, key, default) or "").strip()` の結果を代入する。
8. 条件 `not raw` を判定し、真なら内部処理を行う。
9. `raw` に `str(default or "").strip()` の結果を代入する。
10. `resolve_relative_path(profile_dir, raw)` を返す。

#### 代表コード断片

```
def get_path(cfg: dict, profile_path: str | Path, key: str, default: str = "") -> str:
    profile_dir = Path(profile_path).resolve().parent
    raw = ""
    paths_cfg = cfg.get("paths", {}) if isinstance(cfg, dict) else {}
    if isinstance(paths_cfg, dict):
        raw = str(paths_cfg.get(key, "") or "").strip()
    if not raw:
        raw = str(get_section(cfg, key, default) or "").strip()
    if not raw:
        raw = str(default or "").strip()
    return resolve_relative_path(profile_dir, raw)
```

## 8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

## 9 このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_profile.py` を **config** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。